

Introduction to API Attacks

Application Programming Interfaces (APIs) are foundational to modern software development, with web APIs being a prominent form. They enable seamless communication and data exchange across diverse systems over the internet, facilitating integration and collaboration among different software applications.

At their essence, APIs consist of defined rules and protocols that dictate how disparate systems interact. They specify requirements, delineate access methods for resources, and define expected response structures. APIs can be public, accessible to external parties, or private, restricted to specific organizations or groups of systems.

API Building Styles

Web APIs can be built using various architectural styles, including [REST](#), [SOAP](#), [GraphQL](#), and [gRPC](#), each with its own set of use cases:

- [Representational State Transfer](#) ([REST](#)) is the most popular API style. It uses a stateless protocol where clients make requests to resources on a server using standard HTTP methods ([GET](#), [POST](#), etc.). [RESTful](#) APIs are stateless, meaning each request contains all necessary information for the server to process it, and responses are typically serialized as JSON or XML.
- [Simple Object Access Protocol](#) ([SOAP](#)) uses XML for message exchange between systems. It is highly standardized and offers comprehensive features for security, transactions, and error handling. However, it is generally more complex to implement and use than [RESTful](#) APIs.
- [GraphQL](#) is an alternative style that provides a more flexible and efficient way to query data. Instead of returning a fixed set of fields for each resource, [GraphQL](#) allows clients to request exactly what data they need, reducing over-fetching and under-fetching of data. It uses a single endpoint and a strongly-typed query language to retrieve data.
- [gRPC](#) is a newer style that uses [Protocol Buffers](#) for message serialization, providing high performance and an efficient way to communicate between systems. [gRPC](#) APIs can be developed in various programming languages and are particularly useful for microservices and distributed systems.

In this module, our focus will be on attacks against a [RESTful web API](#). However, the vulnerabilities demon

API2:2023 - Broken Authentication	The authentication mechanisms of the API can be bypassed or circumvented.
API3:2023 - Broken Object Property Level Authorization	The API reveals sensitive data to authorized users that they should not have access to.
API4:2023 - Unrestricted Resource Consumption	The API does not limit the amount of resources users can consume.
API5:2023 - Broken Function Level Authorization	The API allows unauthorized users to perform authorized operations.
API6:2023 - Unrestricted Access to Sensitive Business Flows	The API exposes sensitive business flows, leading to potential financial loss.
API7:2023 - Server Side Request Forgery	The API does not validate requests adequately, allowing attackers to spoof requests with internal resources.
API8:2023 - Security Misconfiguration	The API suffers from security misconfigurations, including vulnerabilities.
API9:2023 - Improper Inventory Management	The API does not properly and securely manage version inventory.
API10:2023 - Unsafe Consumption of APIs	The API consumes another API unsafely, leading to potential security risks.

This module will focus on exploiting all these security risks and understanding how to prevent them.

[Next ➡](#)

+10 Streams

Table of Contents

Introduction

[Introduction to API Attacks](#) Introduction to Lab

OWASP API Security Top 10

 Broken Object Level Authorization Broken Authentication Broken Object Property Level Authorization Unrestricted Resource Consumption

 Start Instance

 / 1 spawns left